유닛 액션 UI 연출 및 타격감(Juice) 시스템 구현 완료 보고서

프로젝트: WildCard (UE 5.7 C++) **작성일시:** 2026-10-13 **상태:** 구현 완료 & 컴파일 검증 성공 (Exit Code 0)

1. 개요 및 배경 (Executive Summary)

기존 WildCard의 소환 유닛 액션(이동, 전투, 효과)은 데디케이트 서버 상에서 규칙 및 수치 계산 위주로 작동하여, 플레이어에게 시각적인 손맛(Game Feel / Juice)과 조작 편의(QoL)를 충분히 제공하지 못했습니다.

- **이동(Move)의 단절감:** 서버에서 `SetActorLocation` 호출 시 클라이언트 뷰포트에서 즉각 순간이동하여 위치 이동감이 결여됨.
- **전투(Combat) 타격감 부재:** 서로의 체력 수치만 즉각 깎일 뿐, 공격 돌진(Lunge)이나 피격 발광(Hit Flash) 등 시각적 인과관계가 전달되지 않음.
- **타겟팅 조작 집중도 결여:** 공격/이동/효과 대상 타일을 고를 때 화면 포커스 및 안내 문구가 없어 조작 실수를 유발할 위험이 존재함.

본 작업에서는 **KISS 원칙(최소 침습 및 기존 컴포넌트 재사용)**과 **UE5 데디케이트 서버 규약**을 엄격히 준수하여, 모든 유닛 액션에 고품질 시각 연출을 완성했습니다.

> **핵심 구현 성과:**
> 1. **타겟팅 UI 일원화:** 이동(초록) / 공격(빨강) / 효과(보라) 타겟팅 시 화면 딤(Dim) + 상단 가이드 텍스트 배너 자동 연계.
> 2. **유닛 도약 이동 연출:** `OnRep_CurrentTileChanged` 기반 0.25초 부드러운 포물선 도약(Arc 35.0f, EaseInOut) 클라이언트 보간 구현.
> 3. **전투 타격 피드백:** `NetMulticast`를 통해 공격자 순간 런지(Lunge 35u) & 시안 발광, 피격자 붉은 점멸 연계.
> 4. **효과 시각 피드백:** 트랩 및 스킬 발동 시 시전 유닛 테두리 발광 연계.

2. 세부 구현 아키텍처 및 코드 변경 내역

1) 타겟팅 UI 연계 (`InGameController.cpp`)

유닛 액션 메뉴에서 행동을 선택했을 때, 최근 구축된 `HUDInstance->ShowTargetingFocus(PromptText)`를 즉시 호출하여 화면을 어둡게 집중시키고 타일 발광을 극대화합니다.

```
void AInGameController::Client_ChangeMoveableTiles_Implementation(const TArray<ATile*>& InTiles)
{
    if (InTiles.Num() == 0) {
        TileSelectComp->SetCurrentState(ETileInteractionState::Idle);
        if (HUDInstance) HUDInstance->HideTargetingFocus();
    } else {
        GetPlayerState<AInGamePlayerState>()->SetOldTiles(InTiles);
        for (ATile* Tile : InTiles) Tile->SetTileState(2); // Green
        if (HUDInstance) HUDInstance->ShowTargetingFocus(FText::FromString(TEXT("이동할 타일을 선택하세요"))
    }
}
```

취소 시 `CancledCanMoveTiles`, `CancledCanCombatTiles`, `CancledCanEffectTiles`에서 `HUDInstance->HideTargetingFocus()`를 호출하여 안전하게 원상 복구됩니다.

2) 유닛 이동 보간 연출 (`UnitBase.h`, `UnitBase.cpp`)

데디케이트 서버 환경에서 서버 액터 위치는 권한(Authority) 동기화를 위해 즉시 반영하되, 클라이언트에서는 `CardVisualRoot`의 상대 좌표(Relative Location)를 역오프셋하여 0.25초 동안 부드럽게 글라이딩 및 점프합니다.

```
void AUnitBase::OnRep_CurrentTileChanged()
{
    if (!IsRunningDedicatedServer() && CurrentTile)
    {
        FVector NewLocation = CurrentTile->GetActorLocation();
        if (!LastKnownTileLocation.IsNearlyZero() && FVector::Dist2D(LastKnownTileLocation, NewLocation) >
        {
            StartVisualMove(LastKnownTileLocation, NewLocation);
        }
        LastKnownTileLocation = NewLocation;
    }
}

void AUnitBase::UpdateVisualMove(float DeltaTime)
{
    VisualMoveAlpha += DeltaTime / FMath::Max(0.01f, VisualMoveDuration);
    if (VisualMoveAlpha >= 1.0f) {
        bIsVisualMoving = false;
        CardVisualRoot->SetRelativeLocation(FVector::ZeroVector);
    } else {
        float SmoothAlpha = FMath::InterpEaseInOut(0.0f, 1.0f, VisualMoveAlpha, 2.0f);
        FVector CurrentOffset = FMath::Lerp(VisualMoveStartOffset, FVector::ZeroVector, SmoothAlpha);
        float ArcOffset = FMath::Sin(VisualMoveAlpha * PI) * VisualMoveArcHeight;
        CurrentOffset.Z += ArcOffset;
        CardVisualRoot->SetRelativeLocation(CurrentOffset);
    }
}
```

3) 전투 타격 및 런지 피드백 (`ChainComponent.cpp`, `UnitBase.cpp`)

서버에서 체력 수치를 교환한 즉시 `Multicast_PlayAttackVisual(Defender)`를 호출하여 양 플레이어 화면에 타격 공방을 재현합니다.

```
// ChainComponent.cpp - UnitAttack 해결 블록
Attacker->SetCardHealth(Attacker->GetCardHealth() - Defender->GetCardAttack());
Defender->SetCardHealth(Defender->GetCardHealth() - Attacker->GetCardAttack());

// [시각 연출] 공격자 런지 + 발광, 피격자 붉은 점멸 네트워크 브로드캐스트
Attacker->Multicast_PlayAttackVisual(Defender);
```

```
// UnitBase.cpp - Multicast 연출 구현
void AUnitBase::Multicast_PlayAttackVisual_Implementation(AUnitBase* TargetDefender)
{
    if (IsRunningDedicatedServer()) return;

    PlayCardActivationEffect(); // 공격자 시안 발광
    if (TargetDefender) {
        TargetDefender->PlayCardHitReaction(); // 피격자 붉은 점멸

        if (CardVisualRoot && !bIsVisualMoving) {
            FVector LungeDir = (TargetDefender->GetActorLocation() - GetActorLocation()).GetSafeNormal2D()
            CardVisualRoot->SetRelativeLocation(LungeDir * 35.0f); // 35u 전진 런지

            FTimerHandle LungeTimer;
            GetWorld()->GetTimerManager().SetTimer(LungeTimer, [this]() {
                if (CardVisualRoot && !bIsVisualMoving) CardVisualRoot->SetRelativeLocation(FVector::ZeroV
            }, 0.15f, false);
        }
    }
}
```

3. 변경 파일 목록 및 컴파일 검증

구분	파일명	주요 변경 사항
헤더	`UnitBase.h`	이동 보간 변수(`VisualMoveDuration` 등) 및 `Multicast_PlayAttackVisual` 등 선언
구현	`UnitBase.cpp`	`OnRep_CurrentTileChanged` 도약 보간 구현, 런지 및 피격 반응 멀티캐스트 구현
서버	`ChainComponent.cpp`	전투 및 트랩 효과 해결 시 멀티캐스트 시각 연출 트리거 훅 연계
컨트롤러	`InGameController.cpp`	Move/Combat/Effect/TacticalSkill 타겟팅 진입 및 취소 시 `Show/HideTargetingFocus` 연동

빌드 검증 결과 (Unreal Build Tool):
- 타겟: `WildCardEditor Win64 Development`
- 상태: **빌드 성공 (Exit Code 0, Error 0, Warning 0)**
- 실행 시간: 13.92초